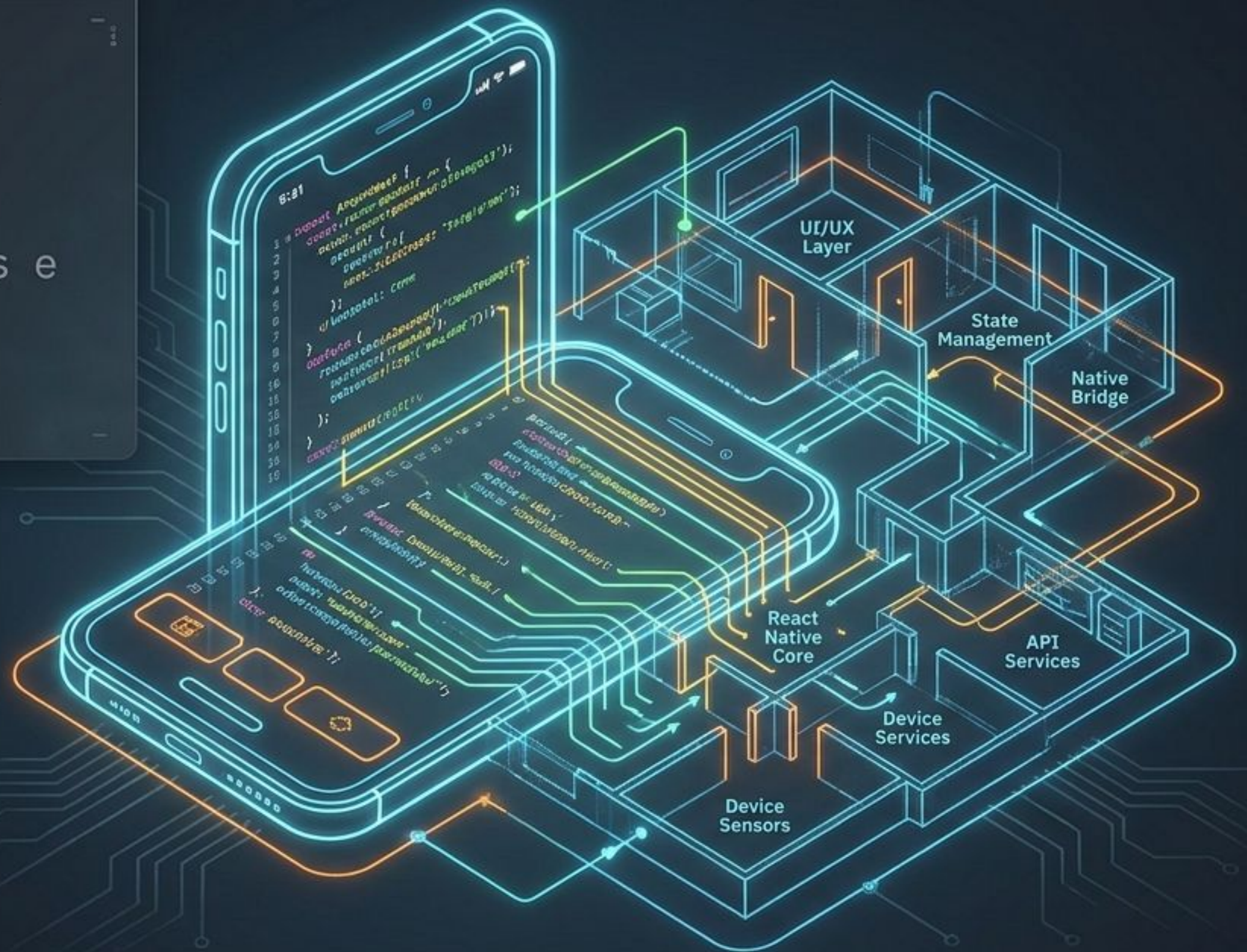


# A Arquitetura do Mobile

// Revisão de Fundamentos e  
Síntese do Ecossistema  
React Native



# A Escolha do Terreno (Nativo vs Híbrido)

## Desenvolvimento Nativo

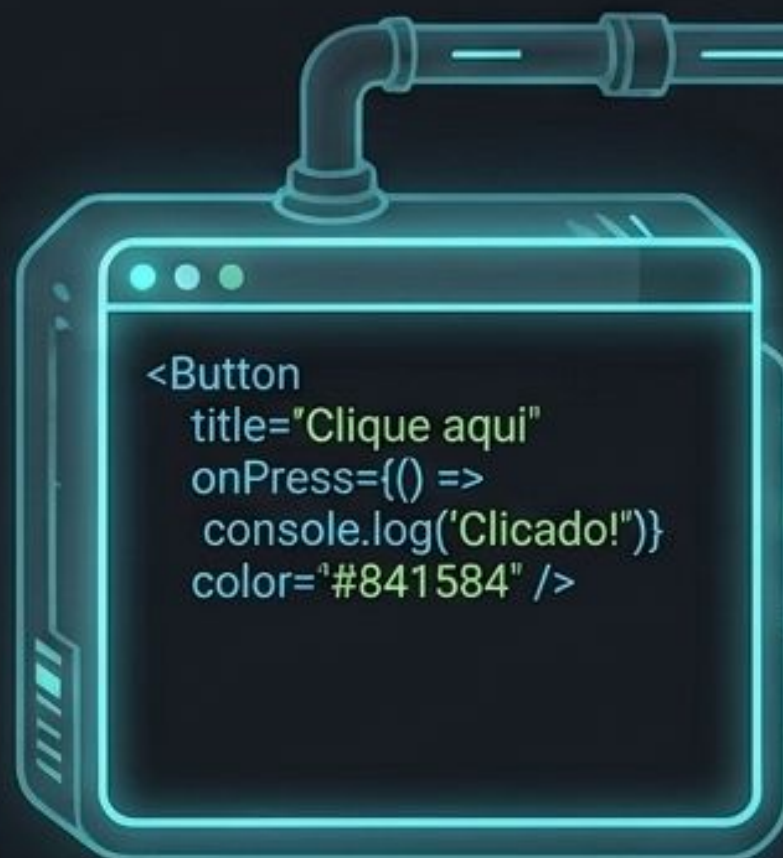
- Linguagens: **Java/Kotlin** (Android), **Objective-C/Swift** (iOS)
- Ferramentas: **Android Studio**, **XCode**
- Vantagem: **Alta performance, acesso total ao hardware**
- Dor: Custo dobrado, código reescrito para cada plataforma

## Desenvolvimento Híbrido (React Native)

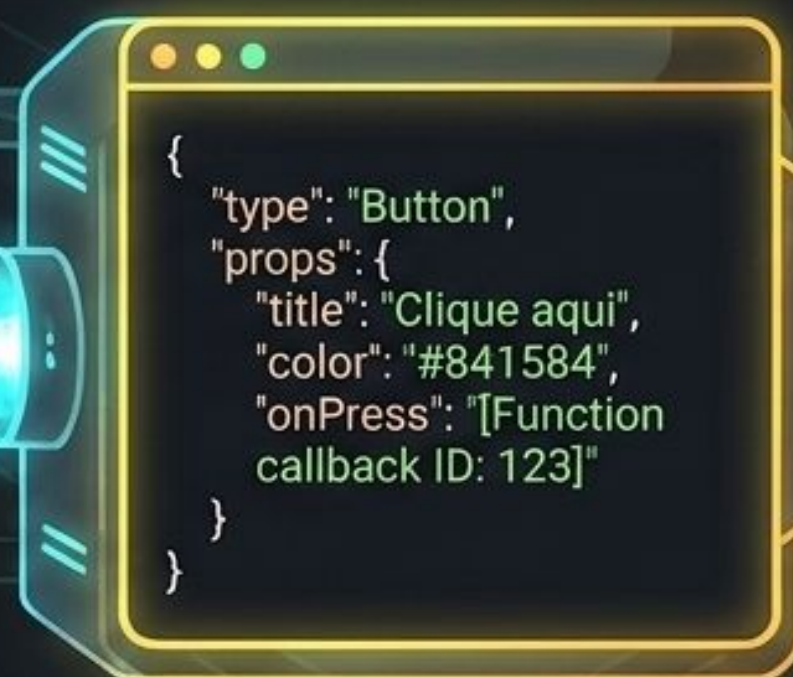
- Linguagem: **JavaScript / TypeScript**
- Ferramentas: **Expo**, **VS Code** (ReactNative app pres....)
- Vantagem: **Código único** (Write once, run anywhere), **alta produtividade**
- Trade-off: Ligeiro overhead de processamento

# A Mágica por Trás da Cortina (The Bridge)

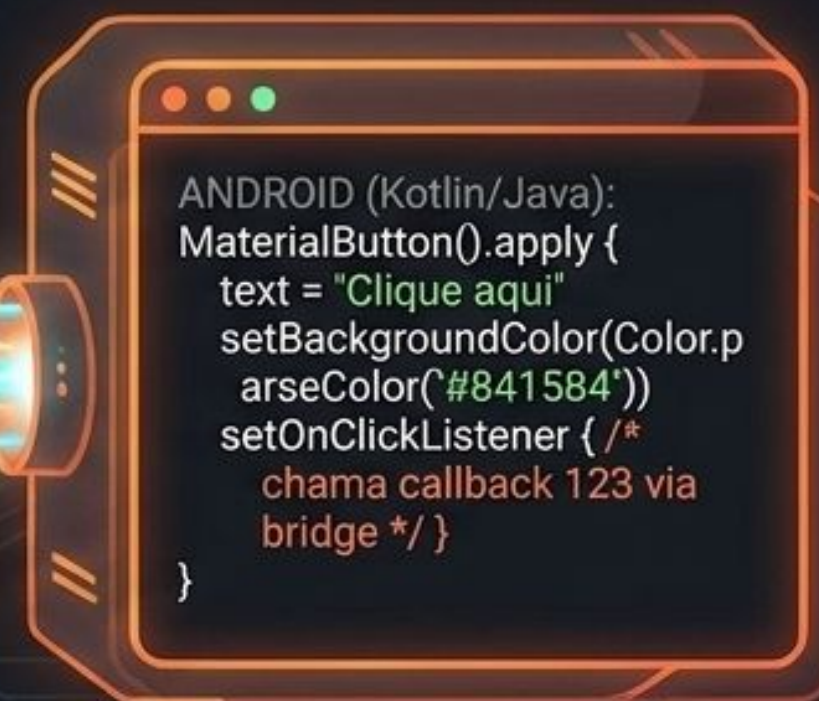
## 1. JavaScript Thread



## 2. The Bridge (Ponte)



## 3. Native Thread



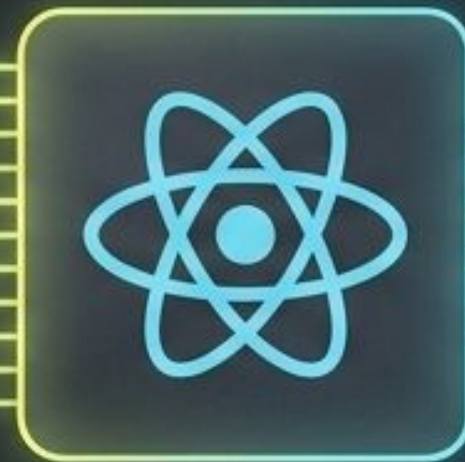
# 0 Ecossistema de Ferramentas

```
export "bootloaderset()" => {  
  const hcvast = 'expt';  
};  
  
export class="eu">  
  <div class="react-nate">  
    <pon select="bomeort zote">{Aicocier />  
  </head>  
  <div>  
    <powt nades="iamp">vluu3ate</apont>  
  </div>  
</div>  
</div>  
}
```



**Node.js**

Motor (Runtime) JS



**React Native / Expo**

Framework e Abstração



**VS Code**

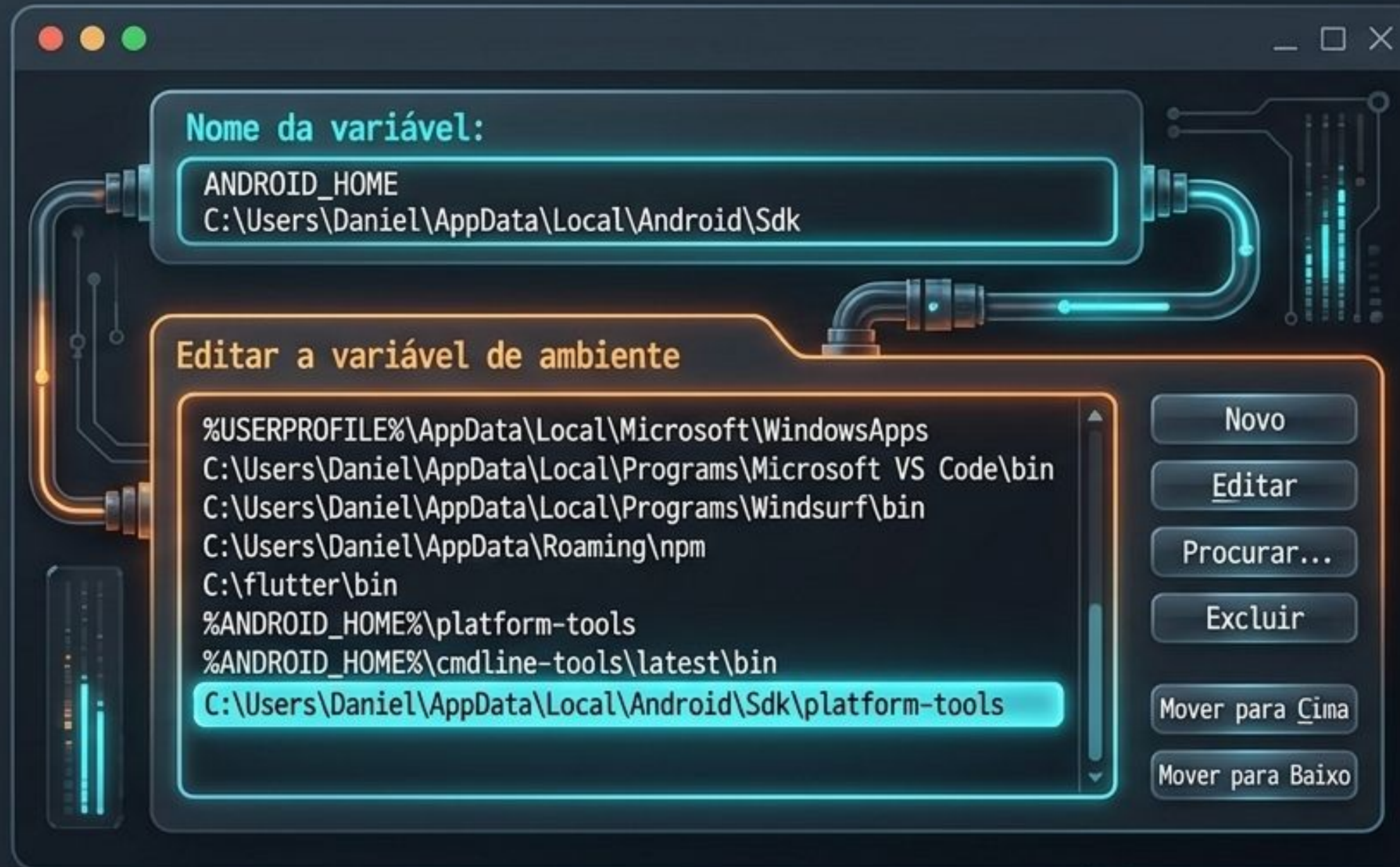
Editor IDE



**Expo Go**

Player Físico  
(Hot Reload)

# Fundações do Emulador (Variáveis de Sistema)



1. **ANDROID\_HOME:**  
Aponta para o SDK.
2. **Path:** Torna o SDK globalmente acessível.
3. **platform-tools:**  
Libera a comunicação IDE -> Emulador.

# As Leis do JSX

```
1 import { Boon } from 'scode';
2
3 export default function App {
4   return (
5     <Image source={img}>
6       <TextInput />
7     <View />
8   )
9 }
```

DON'T



```
1 <Image source={img}>
2
3 <TextInput>
4
5 <View></View> (sem filhos)
```

DO



```
1 <Image source={img} />
2
3 <TextInput />
4
5 <View />
```

source={img}

**Regra 1:** Fechamento  
Obrigatório  
(Self-closing tags)

**Regra 2:**  
camelCase para  
atributos

**Regra 3:** Chaves { }  
para injetar expressões  
JavaScript

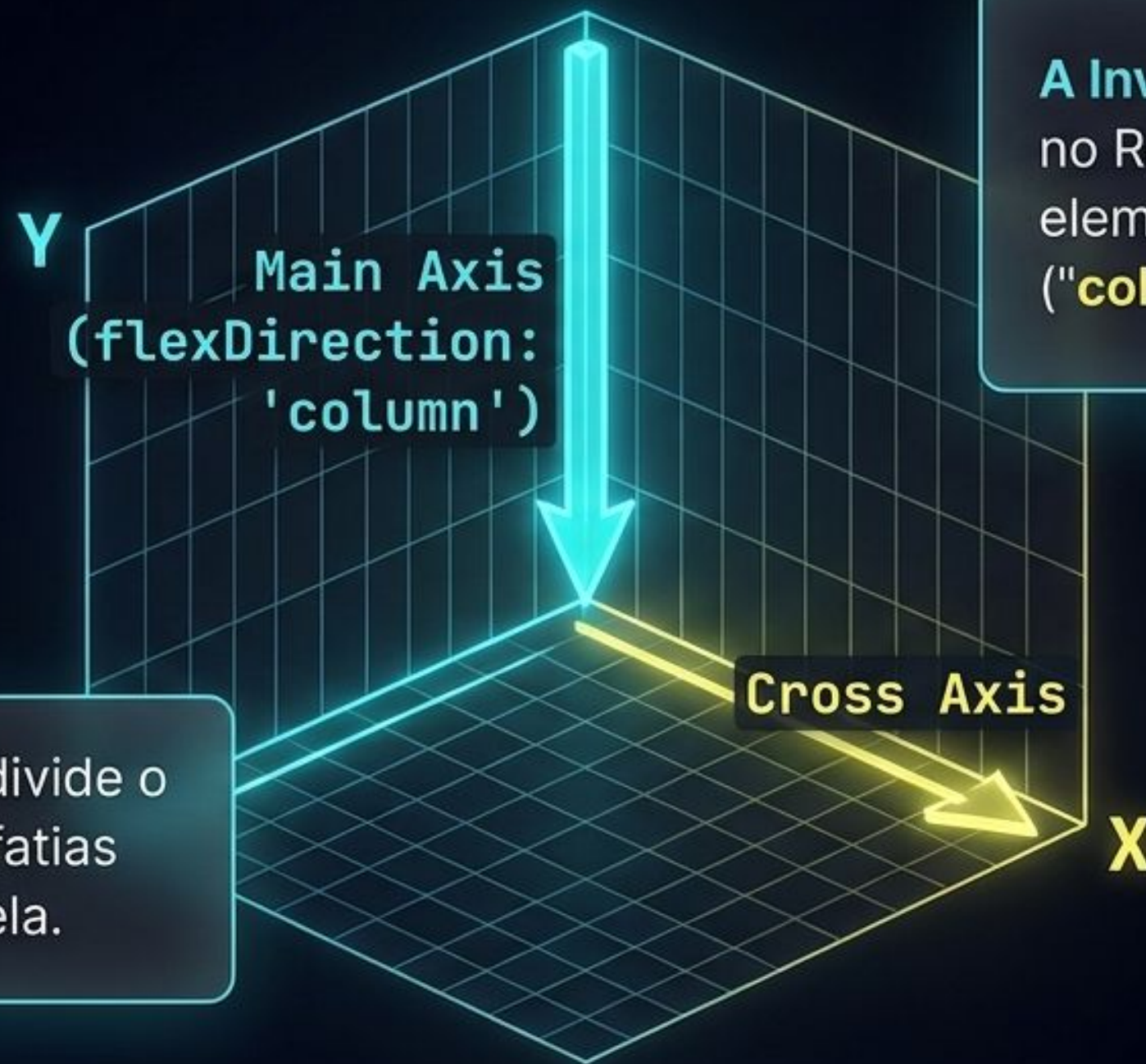
# StyleSheet: O Decorador em JavaScript

| HTML/CSS                      | React Native                    |
|-------------------------------|---------------------------------|
| <code>class='btn'</code>      | <code>style={styles.btn}</code> |
| <code>background-color</code> | <code>backgroundColor</code>    |
| <code>border-radius</code>    | <code>borderRadius</code>       |

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "■#fff",
    alignItems: "center",
    justifyContent: "center",
  },
});
```

Estilos não são arquivos CSS externos, são **OBJETOS JAVASCRIPT** isolados da regra de negócios. As duplas chaves `{{ }}` no inline style representam: 1ª abre expressão JS, 2ª cria objeto anônimo.

# A Bússola do Layout (Eixos do Flexbox)



**A Inversão Mobile:** O padrão no React Native puxa os elementos de cima para baixo ("**column**"), diferente da Web.

**A Proporção:** "**flex: 1**" divide o espaço disponível como fatias matemáticas exatas na tela.

# Dominando a Distribuição de Elementos

## align-content

flex-start



flex-end



center



stretch



space-between

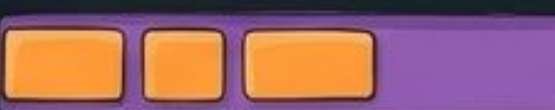


space-around

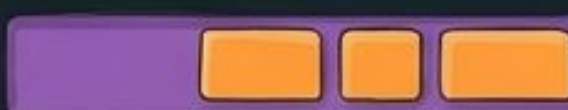


## justify-content

flex-start



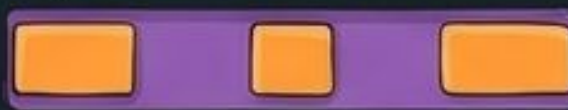
flex-end



center



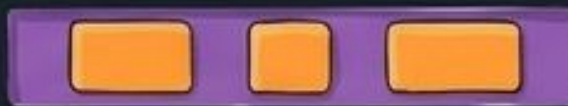
space-between



space-around



space-evenly



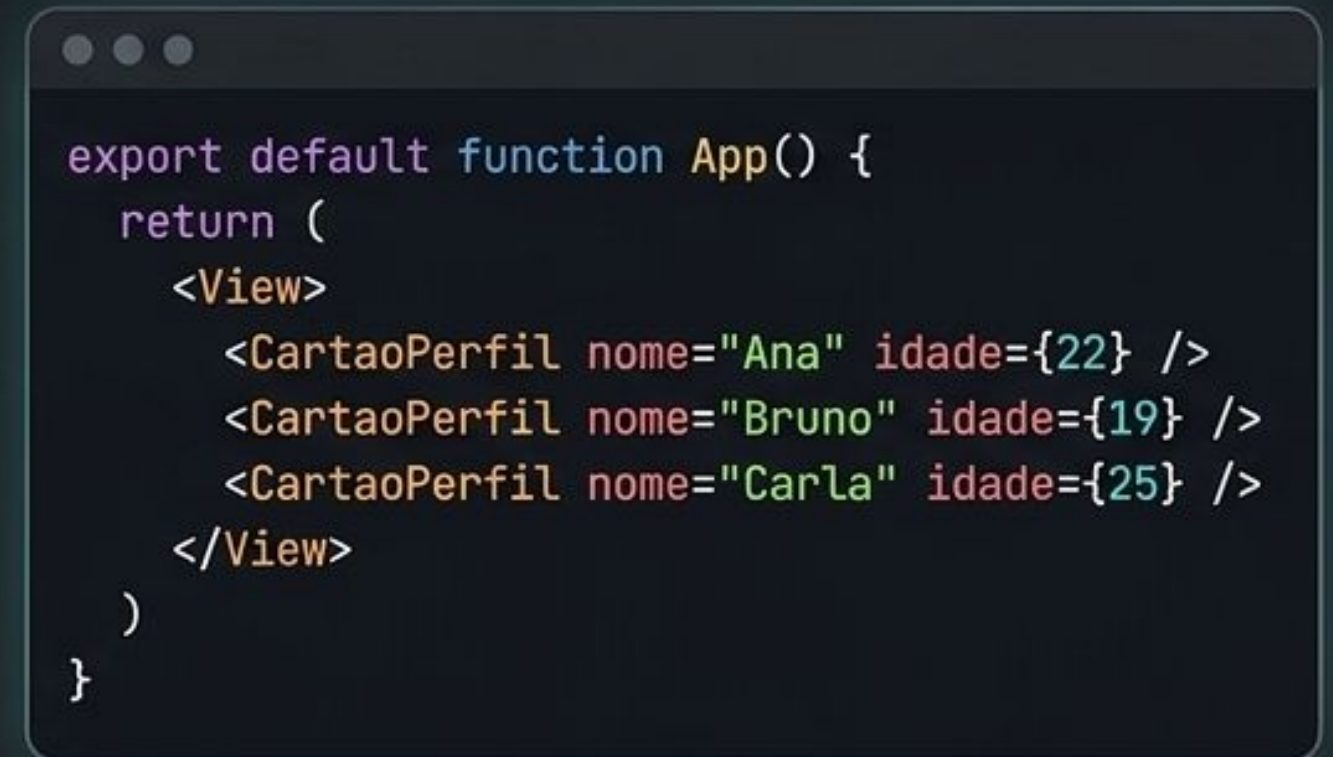
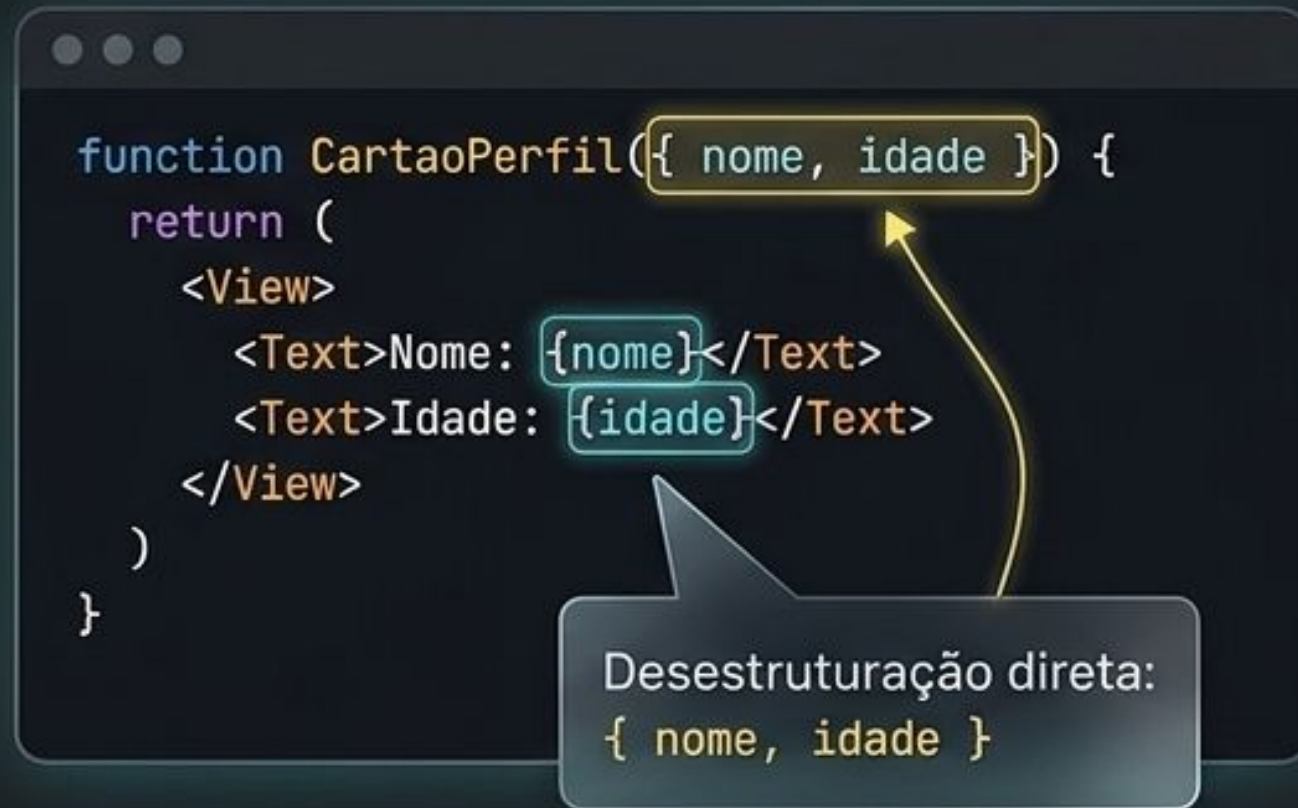
### Padrão Ouro de Centralização

```
// O padrão mais usado: centralizar um elemento na tela inteira
<View style={{
  flex: 1,
  justifyContent: 'center',
  alignItems: 'center',
}}>
  <Text style={{ fontSize: 24 }}>Centralizado!</Text>
</View>
```

# Componentes & Props (O DNA Mutável)

## A Recepção (Filho)

## O Envio (Pai)



# 0 Motor da Tela (useState & useEffect)

## 0 Tanque de Dados (useState)

Armazena estados mutáveis.

Alterar este tanque é a única forma de forçar a tela a se re-desenhar (re-render).



**Regra de Ouro:**  
Hooks APENAS no nível superior (Top Level).  
Nunca dentro de loops ou if/else.

## A Injeção de Efeitos (useEffect)

Engata no ciclo de vida.

Responde silenciosamente às mudanças (nasce, atualiza, morre) baseado no array de dependências.

# 0 Controle Silencioso (useRef)



## Memória Persistente

Guarda uma referência a um valor que sobrevive intacto ao longo de dezenas de renderizações.

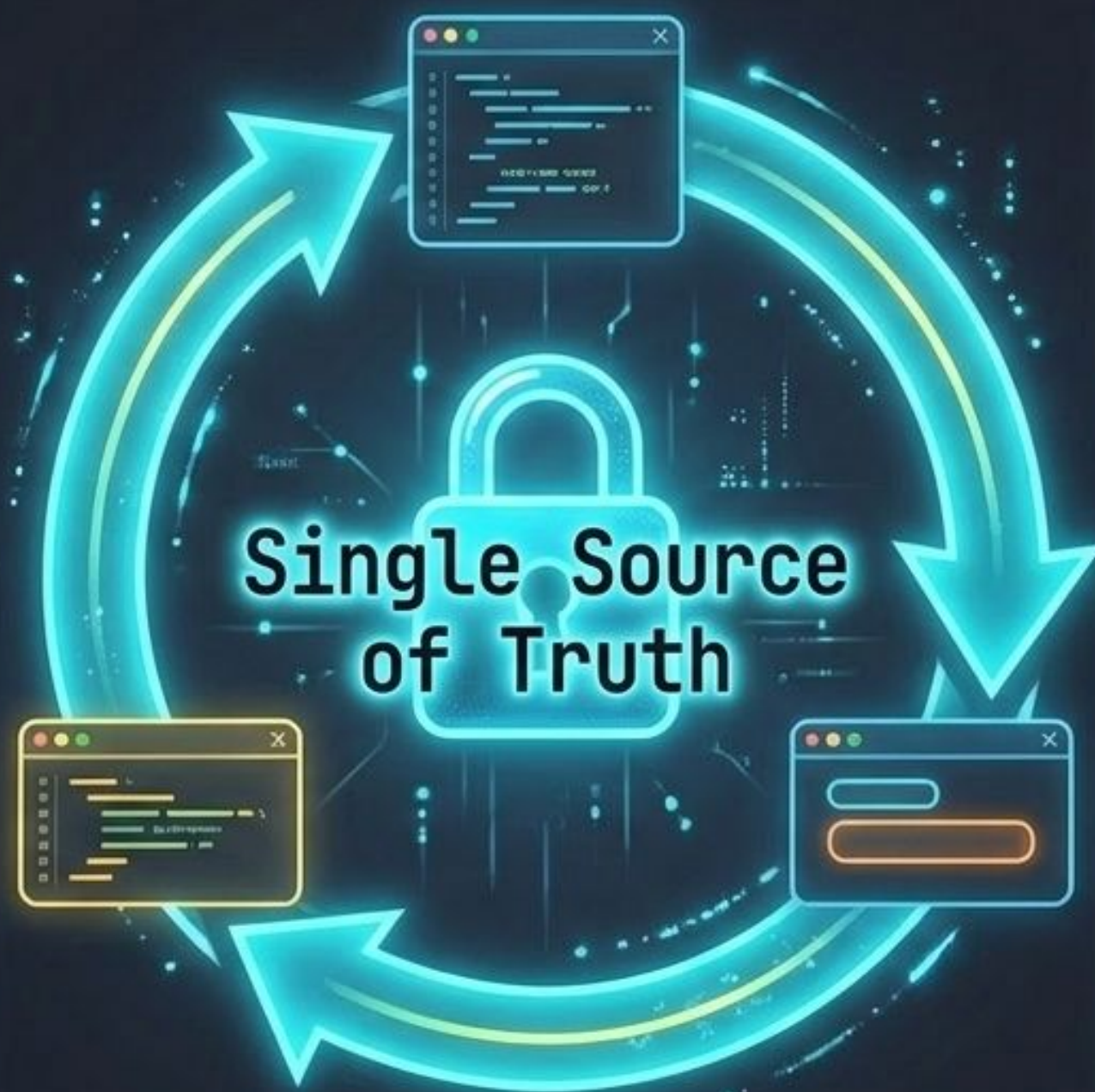
## Zero Re-render

Alterar o valor dentro de `'current'` é invisível para a interface. A tela não pisca, o motor não recalcula.

## Contato Direto

Chave mestre para acessar elementos nativos do DOM/Hardware diretamente (ex: forçar o teclado a abrir num `TextInput`).

# O Fluxo de Dados (Inputs Controlados)



## Passo 1: Leitura

O TextInput exibe o que está no estado:  
``value={estado}``

## Passo 3: Atualização

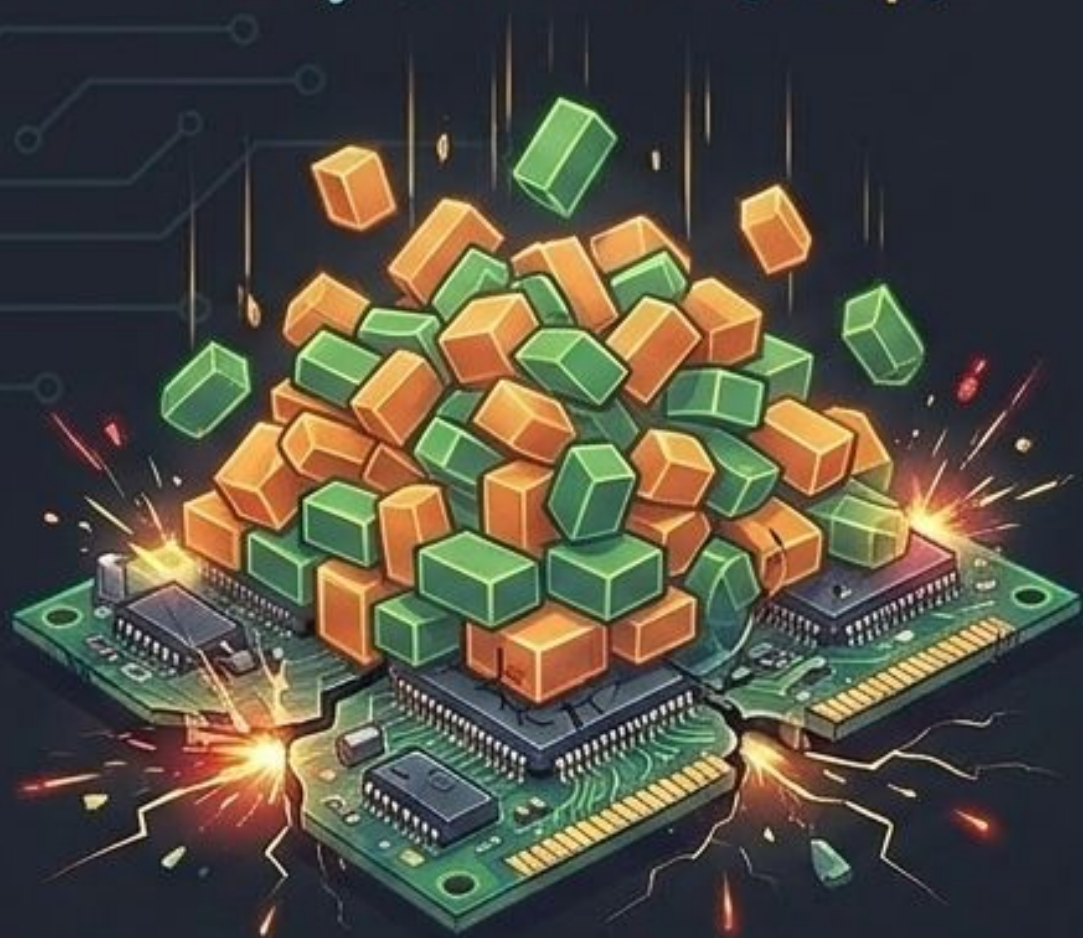
O estado interno muda, forçando o input a exibir a nova letra.

## Passo 2: Escrita

Usuário digita, disparando:  
``onChangeText={(texto) => setEstado(texto)}``

# A Batalha das Listas (.map vs FlatList)

0 Laço Padrão (.map)



Renderiza todos os itens **simultaneamente**.  
**Fatal** para **performance** em feeds longos.

0 Componente Nativo (<FlatList />)



Os 3 Pilares Obrigatórios

- ✓ 1. `data={array}` (A fonte)
- ✓ 2. `keyExtractor` (Identidade única)
- ✓ 3. `renderItem` (0 JSX desenhado)

# O Mapa do Aplicativo (Arquitetura de Navegação)

```
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>
      { /* navegadores ficam aqui dentro */ }
    </NavigationContainer>
  );
}
```

**<NavigationContainer>**  
O Pai de Todos



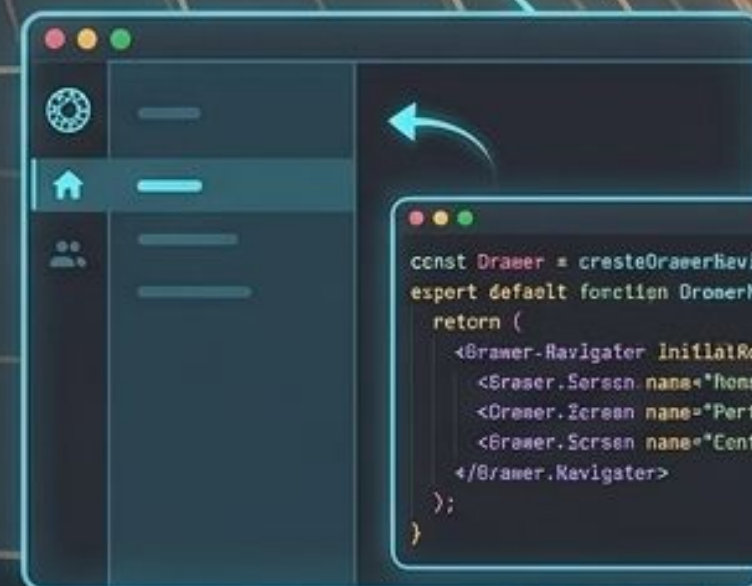
## Stack Navigation

Navegação linear em profundidade.



## Bottom Tab

Divisões primárias fixas (Home, Busca, Perfil).



## Drawer Navigation

Menu lateral expansível para fluxos secundários.

```
const Drawer = createDrawerNavigator();

export default function DrawerNavigator() {
  return (
    <Drawer.Navigator initialRouteName="Home">
      <Drawer.Screen name="Home" component={HomeScreen} />
      <Drawer.Screen name="Perfil" component={PerfilScreen} />
      <Drawer.Screen name="Config" component={ConfigScreen} />
    </Drawer.Navigator>
  );
}
```